Why, What & How

CombatX — real-time competitive coding battles
Document 2 of 3 · Technical Approach · User Guide

WHY

The problem

Competitive programming today has two shapes, and neither one is *a match you can start right now with someone you know.*

Practice sites are single-player

LeetCode, HackerRank, and their peers are asynchronous by design. You pick a problem, you solve it, you get a green check. There is no opponent, no clock that belongs to anyone but you, and no reason to be **first**.

That trains one skill — correctness. But a contest tests a different one: correctness *under pressure*, with someone else's progress visible in the corner of your eye. You cannot practice that alone, and these platforms never ask you to.

Contest platforms are scheduled and serious

Codeforces and CodeChef do have real competition. But rounds start at fixed times, run for two hours, and rank you against thousands of strangers on a rating ladder that follows you around.

That's a genuine sport and it serves people well. It just isn't what a university coding club wants on a Thursday evening, or what two friends want when one says *"bet you can't solve this faster than me."*

The gap

Nothing fills the space between them: **short, spontaneous, head-to-head.** Same problem, both players, one clock, someone wins, done in ten minutes.

Groups improvise it today — a shared screen, a phone timer, and the honor system. It works badly. Someone reads the problem earlier than someone else. Nobody can verify the other person's tests actually passed. There's no record of who won.

Who this is for

Coding communities and events. Bootcamps, university clubs, Discord servers, internal team competitions — groups that already gather to code and want to run a bracket, a warm-up round, or a quick grudge match.

Two properties of that audience drive the entire design:

They gather in bulk, briefly. A club cannot ask forty people to create accounts before a session starts. So play is **guest-only** — type a name, share a room code, you're in. There is no registration, no email, no password. (In fact, there is no password *column* — see Technical Approach §5.)

Their organizer is not a DevOps engineer. Whoever runs the event should not spend an afternoon wiring up a database, a queue, and a code sandbox. So the entire stack is **one command**, with schema, seed data, and language runtime all provisioned automatically on first boot.

What this is *not*

No ratings. No ladders. No matchmaking against strangers. No streak mechanics pulling you back tomorrow.

CombatX assumes **you already know who you're playing.** You're in a room together, physically or on a call. It's a tool for a group that has gathered — not a platform trying to keep you online.

WHAT

In one sentence

Two teams race to solve the **same** problem; pass every hidden test first for an instant win, or hold the highest passed-count when the clock expires.

The three phases

1. Lobby

Players join with a room code and take a seat on Team A or Team B, then ready up. The host can start once both sides are seated, equal in size, and ready.

2. Arena

The server reveals the same problem to both sides simultaneously and starts the clock. Players write code and submit. Each submission runs against the hidden tests in a sandbox, and the verdict streams back.

What you see: your own verdict, in full — which tests passed, what failed.

What your opponent sees: a number. How many tests you've cleared. Nothing else — not your code, not which tests, not your errors.

3. Results

Two ways a battle ends:

Reason	Condition
`ALL_PASSED`	A side passes every test — **immediate win**
`TIMEOUT`	Clock expires — **highest passed-count wins**

Ties on passed-count break by **who got there first**, using the server's receipt timestamp.

Current scope

1v1 works end to end — guest auth, lobby, live scoring, sandboxed judging, both win conditions, persisted results.

2v2 through 4v4 are modelled across the schema, protocol, and game rules, but gated in the UI pending a lobby flow for larger rosters.

Python 3.12 is the shipped language. The runtime list is protocol-driven, so adding more is a config change plus a provisioner entry.

HOW

Three constraints, three decisions

Every significant architecture choice follows from one of three properties of the problem. None of them are technology preferences.

Constraint 1 — A match needs a referee

The moment there's an opponent, every piece of state becomes contestable. Who's ready. Who submitted first. Whose clock it is. Who won.

If any of that lives on the client, it can be forged. A contest you can cheat isn't a contest.

→ `ws-server` **is authoritative.** Clients send *intent*, never facts. A client cannot mark itself ready, award itself a point, set its own submission time, or end the battle.

→ **The rules are pure functions.** `packages/game` decides readiness, scoring, and outcome with no database and no sockets — so the component that determines winners is the one that's easiest to test in isolation. It has unit tests covering both win conditions and the tie-break.

Constraint 2 — A contest requires hidden tests

If you can read the test cases, you can special-case them, and the score means nothing.

→ **Tests never leave the backend.** They live in Postgres and are read only by `judge-worker`. They are never sent to a client, not even the submitter's.

→ **Opponents get a count, not a signal.** The `opponent:progress` frame carries a passed-count and nothing else — enough to feel the race, not enough to reverse-engineer the problem. The schema has no field that *could* carry source code, so this is enforced at compile time rather than by discipline.

Constraint 3 — Untrusted code is dangerous *and* slow

These are two problems, and they need two different answers.

Dangerous → isolate it. Every submission is a stranger's code running on your machine. It executes in a **Piston** sandbox in a separate container, reachable only by the worker — with execution timeouts, an HTTP abort guard, and output truncation so one runaway `print` can't exhaust memory.

Slow → get it off the socket. Execution costs hundreds of milliseconds per test. A game server that blocks on judging stops sending timer ticks *to everyone in the room*. So submissions become **BullMQ jobs on Redis**; `judge-worker` consumes them and publishes verdicts back over pub/sub.

The socket path stays responsive no matter how many people hit Run at once, and judging capacity scales independently of the game server.

The invisible constraint — the two sides must agree

A subtler failure mode: the client and the server disagreeing about the rules *mid-tournament*.

→ **Shared, compiled contracts.** `packages/protocol` (Zod schemas for every frame) and `packages/game` (the rules) are imported by both the frontend and the backend. A protocol change that breaks a client is a **type error at build time**, not a mystery bug during someone's event.

Validation runs at every boundary — REST bodies, socket frames in both directions, and jobs pulled off the queue. Nothing is trusted merely because it arrived over a channel that was trusted a moment ago.

Why one command

The audience constraint from WHY shows up as an engineering requirement: an organizer must be able to host this without a DevOps afternoon.

That's why `docker compose up --build` is genuinely sufficient. Two init containers handle what would otherwise be manual setup — `db-init` applies the schema and seeds problems, `piston-init` installs the language runtime. Both are idempotent and gated behind healthchecks, so the first request can never hit an unmigrated database or an empty sandbox.

It is the difference between a project someone can run and a project someone will actually use at an event.

Summary

The problem	The response
No quick head-to-head format exists	Room-code battles, ten minutes, someone wins
Groups can't onboard 40 people	Guest-only — no accounts, no passwords
Every match state is contestable	Authoritative server; clients send intent only
Readable tests make scores worthless	Hidden tests, backend-only; opponents see a count
Untrusted code is dangerous	Piston sandbox, separate container, hard limits
Untrusted code is slow	Queue-backed judging; socket path never blocks
Client/server can drift	Shared Zod protocol + rules, compiled into both
Organizers aren't DevOps engineers	One command, self-provisioning